

주사위 탑 쌓기 - 풀이

작성자: 박상훈

부분문제 1

두 주사위를 하나의 탑으로 쌓을 수 있으려면, 아래쪽 주사위의 윗면에 적힌 수와 위쪽 주사위의 아랫면에 적힌 수가 같아야 한다.

윗면에 A_i 가 적힌 주사위의 아랫면에는 $7 - A_i$ 가 적혀 있으므로, 두 주사위를 하나의 탑으로 쌓을 수 있는 조건은

$$A_1 + A_2 = 7$$

이다. 이 조건을 만족하면 1, 그렇지 않으면 2를 출력한다.

부분문제 2

윗면에 3이 적힌 주사위의 개수를 C_3 , 윗면에 4가 적힌 주사위의 개수를 C_4 라고 하자.

윗면에 3이 적힌 주사위와 윗면에 4가 적힌 주사위는 번갈아 쌓을 수 있다. 두 개수가 같다면 모든 주사위를 하나의 탑으로 쌓을 수 있다.

두 개수가 다르다면, 더 많이 있는 종류의 주사위가 $|C_3 - C_4|$ 개 더 많다. 하나의 탑에서 한 종류의 주사위는 다른 종류보다 최대 1개 더 많이 사용될 수 있으므로, 필요한 탑의 최소 개수는 $|C_3 - C_4|$ 개이다.

따라서 정답은

$$\max(1, |C_3 - C_4|)$$

이다.

부분문제 3

각 x 에 대해, 윗면에 x 가 적힌 주사위의 개수를 C_x 라고 하자.

같은 탑에 들어갈 수 있는 주사위의 윗면에 적힌 수는 각각 다음 세 쌍 중 하나에 속한다.

$$(1, 6), \quad (2, 5), \quad (3, 4)$$

서로 다른 쌍에 속하는 주사위는 같은 탑에 쌓을 수 없으므로, 각 쌍을 따로 생각할 수 있다.

부분문제의 조건에 따라 C_x 와 C_{7-x} 는 항상 서로 다르다. 부분문제 2와 같은 방법을 각 쌍에 적용하면 정답은

$$|C_1 - C_6| + |C_2 - C_5| + |C_3 - C_4|$$

이다.

부분문제 4

부분문제 3과 마찬가지로 $(1, 6)$, $(2, 5)$, $(3, 4)$ 를 각각 따로 생각한다.

한 쌍에 속한 두 종류의 주사위 개수를 P , Q 라고 하자. 두 개수가 모두 0이면 이 쌍에서는 답을 만들 필요가 없다. 그렇지 않고 두 개수가 같다면, 모든 주사위를 번갈아 쌓아 하나의 탑을 만들 수 있다. 두 개수가 다르다면 필요한 탑의 최소 개수는 $|P - Q|$ 이다.

따라서 각 쌍이 정답에 더하는 값은

$$\begin{cases} 0 & (P = Q = 0), \\ \max(1, |P - Q|) & (\text{그 외}) \end{cases}$$

이다. 세 쌍에 대해 이 값을 모두 더하면 정답을 구할 수 있다.

시간복잡도는 $O(N)$ 이다.

간식 분배 - 풀이

작성자: 박영우

부분문제 1

모든 학생은 정확히 하나의 간식을 좋아하므로, 학생이 방에 들어갔을 때 최대 하나의 간식을 가져간다.

만약 여러 학생이 공통으로 좋아하는 간식이 있다면, 그중 나중에 방에 들어가는 학생은 아무 간식도 가져갈 수 없다. 따라서 이 경우에는 목표를 달성할 수 없다.

그렇지 않다면 모든 학생이 서로 다른 간식을 좋아하므로, 학생들을 어떤 순서로 방에 들여보내더라도 모든 학생이 정확히 하나의 간식을 가져간다.

부분문제 2

이 부분문제에서는 목표를 달성하는 것이 가능할 경우, $1, 2, \dots, N$ 번 학생들을 이 순서대로 방에 들여보내도 목표를 달성할 수 있다.

따라서 방 안에 남은 간식을 관리하면서, 각 학생이 순서대로 방에 들어갈 때 가져가는 간식의 개수를 세면 된다.

부분문제 3

이 부분문제에서 학생들이 방에 들어가는 경우의 수는 최대 $N! \leq 120$ 가지이다. 따라서 각각의 경우에 대해 부분문제 2의 알고리즘을 사용하여 모든 학생이 정확히 하나의 간식을 가져가는지 판정하면 된다.

부분문제 4

$\{1, 2, \dots, N\}$ 의 모든 부분집합 S 에 대해, $dp[S]$ 를 다음과 같이 정의한다.

- 만약 S 에 속한 학생들의 순서를 잘 정하여, 그 순서대로 방에 들여보냈을 때 S 에 속한 각 학생이 정확히 하나의 간식을 가져가도록 할 수 있다면 $dp[S] = 1$ 이다.
- 그렇지 않다면 $dp[S] = 0$ 이다.

모든 S 에 대하여, $dp[S] = 1$ 인 조건은 다음과 같다.

- S 가 비어 있거나,
- S 에서 i 를 제거한 집합 T 가 있어서 $dp[T] = 1$ 이고, i 번 학생이 좋아하는 간식 중 T 에 속한 학생들이 좋아하는 간식이 아닌 것은 정확히 1개이다.

따라서, S 의 모든 부분집합 T 에 대해 $dp[T]$ 를 구했다면 $dp[S]$ 는 $\mathcal{O}(N)$ 에 구할 수 있다.

비트필드를 이용한 다이내믹 프로그래밍을 사용하면 $\mathcal{O}(N2^N)$ 에 풀 수 있다.

부분문제 5

현재 일부 학생들이 이미 방에 들어가 정확히 하나의 간식을 가져갔고, 남아 있는 간식 중 정확히 하나만 좋아하는 학생 i 가 있다고 하자.

현재 상태에서 남은 학생들의 순서를 정하여 모든 학생들이 정확히 하나의 간식을 가져가도록 할 수 있다고 하자. 그 순서에서 i 보다 먼저 들어오는 학생은 i 가 가져갈 유일한 간식을 좋아할 수 없다. 따라서 i 를 그 학생들보다 앞으로 옮겨도 다른 학생들이 가져가는 간식에 영향을 주지 않는다. 그러므로, 만약 현재 상태에서 목표를 달성하는 것이 가능하다면 i 를 바로 들여보내도 답이 존재한다.

따라서 다음과 같은 방법으로 문제를 풀 수 있다.

- 현재 남은 간식 중 정확히 하나를 좋아하는 학생이 없다면 불가능하다고 판정한다.
- 그렇지 않은 경우, 정확히 하나의 간식을 좋아하는 학생을 찾고, 그 학생을 방에 들여보낸다. 만약 그러한 학생이 여러 명일 경우, 어떤 학생을 먼저 들여보내더라도 상관없다.
- 위 과정을 모든 학생이 간식을 가져갈 때까지 반복한다.

부분문제 5의 경우, $N \leq 300$ 이다. 다음과 같이 구현할 수 있다.

- 학생과 간식의 관계를 $N \times N$ 배열로 저장한다. i 번 학생이 j 번 간식을 좋아한다면 i 행 j 열에 1을 저장한다.
- 아직 방에 들어가지 않은 모든 학생의 행을 확인하여, 현재 남은 선호 간식이 정확히 하나인 학생을 찾는다. 그러한 행이 존재하지 않는다면 불가능하다고 판정한다.
- 찾은 행을 i 번 행이라 하고, i 번 학생이 좋아하는 간식 중 현재 유일하게 남아있는 것을 j 번 간식이라 하자. 배열에서 j 열을 모두 지운다.
- 위 과정을 모든 학생이 방에 들어갈 때까지 총 N 번 반복한다.

한 단계에 $\mathcal{O}(N^2)$ 번의 연산이 사용되고, 총 N 번 반복해야 하므로 전체 시간 복잡도는 $\mathcal{O}(N^3)$ 이다.

부분문제 6

부분문제 5의 알고리즘을 최적화할 수 있다. 각 학생 i 에 대해 i 번 학생이 좋아하는 간식 중 현재 방에 남은 것의 개수를 cnt_i 라 하고, 이 배열을 관리한다. 다음과 같이 해결할 수 있다.

- 방에 들어갈 학생을 구하는 것은 아직 방에 들어가지 않은 학생 중 $cnt_i = 1$ 인 학생 i 를 구하면 된다. 이때 i 번 학생이 방에 들어가면 정확히 하나의 간식 j 를 가져가게 된다.
- 이 과정에서 j 번 간식을 좋아하는 학생 k 에 대해, cnt_k 가 1 감소한다. j 번 간식을 좋아하는 모든 학생 k 를 찾아 cnt_k 를 갱신한다.

두 과정 모두 $\mathcal{O}(N)$ 에 처리할 수 있고, 총 N 번 반복하므로 전체 시간 복잡도는 $\mathcal{O}(N^2)$ 이다.

부분문제 7

부분문제 6의 알고리즘에서 추가적인 최적화를 할 수 있다.

- $cnt_i \leq 1$ 인 학생들을 큐(queue) 등의 자료구조를 사용해 저장한다. 어떤 학생이 간식을 가져가서 학생 i 의 cnt_i 가 1 감소했다 하자. 이때 $cnt_i \leq 1$ 인지 검사하고, 만약 그렇다면 i 를 큐에 추가하면 된다.
- j 번 간식을 좋아하는 학생들을 구하는 것은, 처음에 간식 j ($1 \leq j \leq N$) 각각에 대해 j 번 간식을 좋아하는 학생들의 배열을 `std::vector` 등의 동적 배열을 만들어 사용하면 된다. 각 간식은 최대 한 번 제거되므로, 각 배열도 최대 한 번 확인된다.

전체 시간복잡도는 $\mathcal{O}(N + \sum C_i)$ 이다.

게임 - 풀이

작성자: 박영우

각 j ($1 \leq j \leq Q$)에 대하여, j 번째 게임에서 s_j 번 방에 출구가 있는 경우 앨리스가 승리한다. 이러한 경우를 먼저 처리한다. 앞으로의 서술에서는 s_j 번 방에 출구가 없다는 것을 가정한다.

부분문제 1

각 j ($1 \leq j \leq Q$)에 대하여, j 번째 게임에서 $s_j = 1$ 일 경우 앨리스가 승리한다. 그렇지 않을 경우 앨리스가 승리할 조건은 다음과 같다.

- $c_1, c_2, \dots, c_{s_j-1}$ 의 최솟값이 k_j 보다 크거나 같다.

위 조건이 성립하면, 앨리스는 1번 방에 도착하기 전까지 현재 방에서 번호가 정확히 1 줄어드는 방으로 향하는 통로를 k_j 개 선택할 수 있다. 따라서 앨리스가 승리한다.

그렇지 않을 경우, $c_t < k_j$ 이며 $1 \leq t < s_j$ 인 정수 t 가 존재한다. 이 경우 밥은 앨리스가 $t+1$ 번 방에 도달하기 전까지는 아무 통로를 선택하다가, $t+1$ 번 방에 도달했을 때 $t+2$ 번 방으로 향하는 통로를 선택하면 된다. (만약 $t+1 = N$ 이라면, 앨리스는 다른 방으로 이동할 수 없으므로 승리할 수 없다) 그러면 앨리스는 t 번 방으로 이동할 수 없고, s_j 번 방에서 시작하여 1번 방에 도달하기 위해서는 반드시 t 번 방을 거쳐야 하므로 앨리스는 승리할 수 없다.

$i = 1, 2, \dots, N-1$ 에 대해 $\min(c_1, c_2, \dots, c_i)$ 를 $\mathcal{O}(N)$ 에 전처리해 두면 각각의 게임에 대해 앨리스가 승리하는지 $\mathcal{O}(1)$ 에 판정할 수 있다.

부분문제 2

이 부분문제에서 1번 방은 다른 모든 방과 통로로 직접 연결되어 있고, 1번 방이 아닌 다른 두 방을 연결하는 통로는 없다.

1번 방에 출구가 있는 경우와 없는 경우로 나눠서 해결한다.

1번 방에 출구가 있는 경우, 각 게임에서 앨리스가 승리할 수 있다면 1번 방에 도달할 수 있다. 따라서 각 게임에 대해 1번 방에 도달할 수 있는지만 검사해도 충분하다.

1번 방에 출구가 없었다 하자. j 번째 게임에 대해 앨리스가 출구가 있는 방에 도달할 수 있다면, 우선 1번 방에 도달할 수 있어야 한다. 우선 이 조건을 검사하고, 성립하지 않는다면 앨리스가 패배한다고 판정한다.

1번 정점에 앨리스가 있고, 이 시점에서 앨리스가 승리할 수 있다면 정확히 한 번의 이동을 통해 출구가 있는 방에 도달할 수 있어야 한다. 그렇지 않다면, 앨리스는 1번 방이 아닌 다른 방으로 오게 되고, 앨리스는 다시 1번 방으로 돌아와야 한다. (혹은 해당 방에 연결된 통로의 수가 적어 더 이상 이동할 수 없다) 밥은 통로를 잘 선택하여 앨리스가 1번 방과 출구가 없는 다른 방을 무한히 왕복하도록 하게 할 수 있고, 따라서 앨리스는 승리할 수 없다.

따라서 1번 방에서 정확히 한 번의 이동을 통해 출구가 있는 방으로 이동할 수 있는지 판정하면 된다. 앨리스가 출구가 없는 방으로 향하는 통로를 고른다면, 밥은 그 통로를 선택하여 앨리스가 출구가 없는 방으로 이동하게 할 수 있다. 따라서 앨리스는 1번 방에 있을 때 출구가 있는 방으로 향하는 통로만을 골라야 한다. j 번째 게임에서 앨리스가 1번 방에 왔을 때, k_j 가 그러한 통로의 수보다 많다면 승리할 수 없고, 그렇지 않다면 승리할 수 있다.

부분문제 3

각 게임에 대해 $k_j = 1$ 이므로, 앨리스는 각 게임에서 정확히 하나의 통로를 고른다. 따라서 밥의 선택과 무관하게 앨리스는 매 이동에서 원하는 인접한 방으로 이동할 수 있다.

따라서, 앨리스가 j 번 게임에서 이길 필요충분조건은 s_j 번 방과 출구가 있는 방이 통로를 통해 연결되어 있는 것이다.

방과 통로들을 그래프로 해석하면, 출구가 있는 방과 s_j 번 방이 같은 연결 요소(connected component)에 있는 것이다.

처음에 주어진 방들을 dfs나 Union-Find 등의 알고리즘을 사용해 같은 연결 요소에 속한 방끼리 분류하고, 각 연결 요소에 출구가 있는지 구하면 쿼리를 $\mathcal{O}(1)$ 의 시간복잡도에 처리할 수 있다.

전체 시간복잡도는 $\mathcal{O}(N + M + Q)$, Union-Find를 사용할 경우 $\mathcal{O}((N + M + Q)\alpha(N))$ 이다.

부분문제 4

양의 정수 k 에 대해 V_k 를 다음과 같이 정의한다.

- 앨리스가 게임에서 k 개의 통로를 선택할 수 있다고 할 때, 앨리스가 게임에서 승리하게 되는 시작 방 번호의 집합은 V_k 이다.

즉, 각 j 에 대해 $s_j \in V_k$ 라면 앨리스가 승리하고, 그렇지 않으면 패배한다.

양의 정수 k 에 대해, 다음과 같은 관찰을 할 수 있다.

- v 번 방에 출구가 있다면 $v \in V_k$ 이다.
- v 와 연결된 통로 중, V_k 에 있는 방으로 향하는 통로가 k 개 이상이라 하자. 그렇다면 $v \in V_k$ 이다.

따라서, V_k 를 구하기 위해 다음과 같은 알고리즘을 생각할 수 있다.

1. 집합 S 를 공집합으로 초기화한다. 각 방 v 에 대해, d_v 를 v 와 인접한 통로 중 S 에 있는 방으로 향하는 통로의 개수로 정의한다. 처음에 d_v 는 모두 0이다.
2. 출구가 있는 방 v 에 대하여 v 를 S 에 추가한다. 이때 v 와 c 개의 통로로 인접한 방 u 에 대해 d_u 에 c 가 더해진다. v 와 인접한 모든 방 u 에 대해 d_u 를 갱신한다.
3. 만약 어떤 방 v 에 대해 v 가 S 에 없고, $d_v \geq k$ 라면 v 를 S 에 추가한다. 전과 같은 규칙으로 v 와 인접한 방 u 들에 대해 d_u 를 갱신한다.
4. 이 과정을 S 에 없는 모든 방 v 에 대해 $d_v < k$ 가 될 때까지 반복한다.

두 번째 과정이 끝난 이후, S 는 출구가 있는 방의 집합이다. 따라서 이 시점에서 $S \subseteq V_k$ 이다.

세 번째 과정에서 S 에 추가된 모든 v 에 대해 $d_v \geq k$ 이다. 즉, v 와 연결된 통로 중 v 가 추가되기 직전 시점의 S 에 속하는 방과 연결된 것이 k 개 이상이다. 따라서 v 를 추가하기 직전 $S \subseteq V_k$ 였다면, v 를 추가한 이후에도 $S \subseteq V_k$ 이다.

따라서 위 알고리즘이 종료되었을 때, $S \subseteq V_k$ 이다.

이제 $V_k \subseteq S$ 를 증명한다. S 에 없는 방 v 가 V_k 에도 없음을 보이면 충분한데, 이는 S 에 없는 방 v 에 대하여 앨리스가 v 에서 시작했을 때 승리할 수 없음을 의미한다. 이는 다음과 같이 증명할 수 있다.

- v 번 방과 연결된 통로가 k 개 미만일 경우, 앨리스는 다른 방으로 이동할 수 없는데 v 번 방에는 출구가 없으므로($v \notin S$) 앨리스가 승리할 수 없다.
- 그렇지 않을 경우, v 번 방과 연결된 통로 중 S 에 속하는 방으로 향하는 것은 k 개 미만이다. 따라서, 앨리스가 어떻게 통로를 선택하더라도, 그 통로 중 적어도 하나는 S 에 없는 방으로 향한다.
- 밥이 그러한 통로를 반복해서 선택하는 전략을 사용할 경우, 앨리스가 어떻게 이동하더라도 S 에 속한 방으로 이동할 수 없다. 출구가 있는 모든 방은 S 에 속하므로, 앨리스는 출구가 있는 방에 도달할 수 없어 승리할 수 없다.

따라서, $V_k = S$ 이다.

이 부분 문제에서는 모든 게임에 대해 k_j 가 동일하므로, 한 V_k 만 구하면 충분하다. 앞서 제시한 알고리즘을 그대로 사용할 수 있는데, d_u 가 k 이상이며 S 에 속하지 않는 방 u 들을 큐(queue) 등의 자료구조를 사용하여 관리하면 충분하다.

위 알고리즘에서 방 u 가 S 에 추가되었을 때, u 와 연결된 통로들이 한 번 순회된다. 따라서 각 $i(1 \leq i \leq M)$ 에 대해, 방 a_i 와 b_i 를 연결하는 통로들은 최대 두 번 확인된다.

전체 시간복잡도는 $\mathcal{O}(N + M + Q)$ 이다.

부분문제 5

이 부분문제에서는 모든 게임에 대해 앨리스가 시작하는 방 s_j 가 같다. $s_1 = s_2 = \dots = s_Q = s$ 라 하자.

다음과 같은 관찰을 할 수 있다.

- 모든 양의 정수 i 에 대하여, $V_{i+1} \subseteq V_i$ 이다.

부분문제 4의 알고리즘에서 S 에 정점이 추가되는 조건은 k 가 증가함에 따라 더 강해지는데, 따라서 $k = i + 1$ 일 때 알고리즘에서 v 가 S 에 추가되었다면 $k = i$ 일 때에도 추가된다.

따라서, $s \in V_k$ 인 최대 k 가 존재한다면, $1 \leq k' \leq k$ 인 모든 k' 에 대해 $s \in V_{k'}$ 이다.

부분문제 4의 알고리즘을 사용하여 한 k 에 대해 V_k 를 구할 수 있으므로, 이분 탐색으로 $s \in V_k$ 인 k 를 찾거나, 존재하지 않는지 판정할 수 있다.

이후 각 게임 j 에 대해서 k_j 와 k 의 대소를 비교하여 앨리스가 승리하는지 판정할 수 있다.

모든 게임에서 주어지는 k_j 중 최댓값을 X 라 하자. $k = 1, 2, \dots, X$ 범위에 대해서만 이분 탐색을 적용해보면 충분하므로, 전체 시간복잡도는 $\mathcal{O}((N + M) \log X + Q)$ 이다.

부분문제 6

이 부분문제에서 $N, M, k_j \leq 3000$ 이다. 따라서 $X \leq 3000$ 이다.

양의 정수 $k = 1, 2, \dots, X$ 에 대해 V_k 를 각각 구하면, 모든 게임에 대하여 앨리스가 승리하는지 $\mathcal{O}(1)$ 에 판정할 수 있다.

전체 시간복잡도는 $\mathcal{O}((N + M)X + Q)$ 이다.

부분문제 7

출구가 없는 각 방 v 에 대해, $v \in V_k$ 를 만족하는 최대 k 를 K_v 라 하자. 그러한 k 가 없다면 $K_v = 0$ 으로 둔다. 또한, v 에 출구가 있다면 $K_v = 10^{18}$ 로 둔다. 그러면 각 게임 j 에서 엘리스가 승리할 필요충분조건은 $k_j \leq K_{s_j}$ 이다.

모든 K_v 를 한 번에 구하기 위해 부분문제 4의 알고리즘을 k 가 큰 값에서 작은 값으로 감소시키며 실행한다.

현재까지 V_k 에 포함되었다고 판정한 방들의 집합을 S 라 하자. S 에 포함되지 않은 각 방 v 에 대해, d_v 를 v 와 S 에 속한 방들을 연결하는 통로의 수로 정의한다.

처음에는 출구가 있는 방들을 모두 S 에 넣고, 현재 k 를 10^{18} 로 둔다.

현재 k 에 대해 $d_v \geq k$ 인 방 v 가 있다면, 부분문제 4의 관찰에 따라 v 도 V_k 에 포함된다. 따라서 이러한 방을 S 에 추가하고, 인접한 방들의 d 값을 갱신한다. 새롭게 조건을 만족하는 방이 생길 수 있으므로, 그러한 방들을 큐를 사용하여 관리하고, 더 이상 생기지 않을 때까지 반복한다.

더 이상 S 에 추가할 방이 없다고 하자. 이때 S 에 속하지 않은 방들의 d_v 중 최댓값을 x 라 하면, $x < k' \leq k$ 인 모든 k' 에 대해 $V_{k'} = S$ 이다. 또한 $d_v = x$ 인 방들은 $k = x$ 일 때 처음으로 V_k 에 포함될 수 있다.

따라서 현재 S 에 속하지 않은 방 중 d_v 가 가장 큰 방을 찾아, 현재 k 를 그 값으로 낮추고 다시 같은 과정을 반복하면 된다. 이때 새롭게 S 에 추가되는 방 v 에 대해 $K_v = k$ 로 둔다.

S 에 속하지 않은 방 중 d_v 가 가장 큰 방을 빠르게 찾는 것은 우선순위 큐 등을 사용하여 처리할 수 있다. d_v 가 갱신될 때마다 (d_v, v) 를 우선순위 큐에 삽입하고, d_v 가 가장 큰 방을 찾을 때에는 우선순위 큐에서 가장 큰 값을 꺼내면 된다. 이때 이미 S 에 포함된 방에 대한 원소는 무시하고, v 가 S 에 포함되지 않는 (d_v, v) 가 나올 때까지 반복하면 된다.

$K_v = 0$ 인 방의 경우, d_v 가 한 번도 갱신되지 않으므로 위 과정에서 처리되지 않는다. 이러한 방들에 대해서는 따로 처리한다.

각 방은 S 에 한 번만 추가되고, 각 통로 정보는 양 끝 방이 추가될 때에만 필요하므로 최대 두 번 확인된다. 이로 인해 우선순위 큐에는 $\mathcal{O}(M)$ 개의 원소가 삽입된다.

따라서 전체 시간복잡도는 $\mathcal{O}((N + M) \log(N + M) + Q)$ 이다.

수열 연산 - 풀이

작성자: 윤교준

수열 B 에 등장하는 원소를 대표 원소라고 하자. 대표 원소는 최종적으로 수열 X 에 남아 있어야 하므로, 합치기 연산에 의해 없어져서는 안 된다.

값 x 가 수열 A 와 B 에서 등장하는 위치를 각각 $\text{pos}_A(x)$, $\text{pos}_B(x)$ 라고 하자. 여기서, $\text{pos}_B(x)$ 는 x 가 대표 원소일 때에만 정의한다.

먼저 두 연산의 성질을 살펴보자.

합치기 연산은 인접한 두 원소 중 큰 원소를 없애는 연산이다. 또한, 교환 연산으로 두 값 $x < y$ 의 순서가

$$[\dots, x, y, \dots] \rightarrow [\dots, y, x, \dots]$$

와 같이 바뀌면, 이 두 원소의 순서를 다시 원래대로 되돌릴 수 없다. y 가 x 보다 앞에 있는 상태에서는 $y < x$ 가 성립하지 않기 때문이다.

따라서, 두 원소의 상대적인 순서는 최대 한 번만 바뀔 수 있다. 즉, 유효한 연산열에 대해 다음이 성립한다.

$$Q \leq \binom{N}{2} + (N - M) < N^2$$

부분문제 1

현재 수열 전체를 하나의 상태로 생각하여 너비 우선 탐색을 수행할 수 있다.

현재 수열의 길이가 K 라면,

- $X_i < X_{i+1}$ 인 각 위치 i 에 대해 교환 연산을 수행한다.
- $K > M$ 이라면, 각 위치 i 에 대해 합치기 연산을 수행한다.

현재 수열의 길이가 M 이 된 뒤에도 교환 연산은 계속 수행할 수 있음에 유의해야 한다.

각 상태를 처음 방문하게 만든 이전 상태와 연산을 저장하면, 수열 B 에 도달했을 때 연산열을 역추적할 수 있다.

$N \leq 8$ 이므로 가능한 상태를 모두 탐색해도 충분하다.

부분문제 2

최종 수열의 길이는 1이다.

교환 연산은 수열의 원소를 없애지 않는다. 합치기 연산을 반복하여 하나의 원소만 남기면, 남는 값은 처음 수열에 있던 모든 원소의 최솟값이다.

수열 A 는 $1, 2, \dots, N$ 의 순열이므로, 마지막에 남은 값은 반드시 1이다. 즉, $B_1 = 1$ 이 성립해야 한다.

$B_1 = 1$ 이라면, 인접한 두 원소를 임의의 순서로 계속 합치면 된다. 정확히 $N - 1$ 번의 합치기 연산이 필요하다.

전체 시간 복잡도는 $\mathcal{O}(N)$ 이다.

부분문제 3

$M = N$ 이므로 합치기 연산을 한 번도 사용하면 안 된다.

두 대표 원소 p, q 가 다음 조건을 만족한다고 하자.

$$\text{pos}_A(p) < \text{pos}_A(q), \quad p > q$$

두 원소의 순서를 바꾸려면, 어떤 순간에 p 와 q 가 인접한 상태에서 교환 연산을 수행해야 한다. 그러나 $p > q$ 이므로 그러한 교환 연산은 불가능하다.

따라서 반드시

$$\text{pos}_B(p) < \text{pos}_B(q)$$

여야 한다.

이 조건은 충분하기도 하다.

현재 수열에서 인접한 두 원소 p, q 가 이 순서로 놓여 있지만, 수열 B 에서는 q 가 p 보다 앞에 있어야 한다고 하자. 만약 $p > q$ 라면 앞에서 설명한 조건에 모순이다. 따라서 반드시 $p < q$ 이고, 두 원소에 교환 연산을 수행할 수 있다.

그러므로, 각 원소 x 의 정렬 기준을 $\text{pos}_B(x)$ 로 두고 버블 정렬이나 삽입 정렬을 수행하면 된다. 정렬 도중 필요한 모든 교환 연산은 유효하다.

전체 시간 복잡도는 $\mathcal{O}(N^2)$ 이다.

부분문제 4

처음 수열은

$$A = [1, 2, \dots, N]$$

이다.

값 1은 다른 어떤 원소보다도 작으므로 합치기 연산으로 없앨 수 없다. 따라서 1은 반드시 대표 원소여야 한다.

반대로 1이 대표 원소라면 항상 가능하다.

수열을 왼쪽에서 오른쪽으로 보면서, 대표 원소가 아닌 값 x 를 만날 때마다 x 의 바로 왼쪽에 있는 원소와 합친다. 현재 x 의 왼쪽에 있는 원소는 항상 x 보다 작으므로, 이 합치기 연산으로 x 만 없어진다.

모든 대표 원소가 아닌 값을 없앤 뒤에는, 부분문제 3과 같은 방법으로 남은 대표 원소들을 수열 B 의 순서대로 정렬하면 된다.

전체 시간 복잡도는 $\mathcal{O}(N^2)$ 이다.

부분문제 5

대표 원소가 아닌 값이 정확히 하나 존재한다. 그 값을 x 라고 하자.

먼저 대표 원소들 사이에는 부분문제 3에서 설명한 조건이 성립해야 한다.

x 의 오른쪽에 x 보다 작은 값이 존재한다고 하자. x 의 오른쪽에서 처음 등장하는 x 보다 작은 값을 z 라고 하자. x 와 z 사이의 모든 원소는 x 보다 크다.

따라서 x 를 오른쪽으로 계속 교환하여 z 의 바로 앞으로 옮긴 뒤, x 와 z 를 합치면 x 를 없앨 수 있다.

이제 x 가 자신의 오른쪽에 있는 모든 원소보다 작다고 하자. 이러한 원소를 수열 A 의 접미사 최솟값이라고 부르자.

즉, $x = A_i$ 가 접미사 최솟값이라는 것은

$$A_i < A_j \quad (i < j \leq N)$$

을 의미한다.

x 가 접미사 최솟값이라면, x 를 없애는 데 사용되는 더 작은 원소는 처음에 x 의 왼쪽에 있어야 한다.

x 의 왼쪽에 있는 대표 원소 중 수열 B 에서 가장 뒤에 등장하는 원소를 y 라고 하자.

대표 원소들을 수열 B 의 순서대로 정렬하면, y 가 x 의 바로 왼쪽에 오게 된다. 따라서 $y < x$ 라면 y 와 x 를 합쳐서 x 를 없앨 수 있다.

반대로 y 가 존재하지 않거나 $y > x$ 이면 불가능하다.

전체 시간 복잡도는 $\mathcal{O}(N^2)$ 이다.

부분문제 6

수열 B 가 수열 A 의 부분수열이므로, 대표 원소들의 상대적인 순서는 처음부터 수열 B 와 같다.

따라서 대표 원소 사이에 교환 연산을 수행할 필요가 없으며, 부분문제 3에서 설명한 조건은 항상 성립한다.

대표 원소가 아닌 접미사 최솟값 x 를 생각하자.

x 의 왼쪽에 있는 대표 원소 중 수열 B 에서 가장 뒤에 등장하는 원소는, 단순히 수열 A 에서 x 의 왼쪽에 가장 가까이 있는 대표 원소이다. 이를 y 라고 하자.

따라서 모든 대표 원소가 아닌 접미사 최솟값 x 에 대해,

- x 의 왼쪽에 대표 원소 y 가 존재하고,
- $y < x$

여야 한다.

접미사 최솟값이 아닌 대표 원소가 아닌 값들은 오른쪽의 더 작은 원소를 이용하여 없앨 수 있다.

그 뒤 남은 접미사 최솟값들은 바로 왼쪽의 대표 원소와 합치면 된다.

전체 시간 복잡도는 $\mathcal{O}(N^2)$ 이다.

핵심 조건

일반적인 입력에서 수열 A 를 수열 B 로 바꿀 수 있을 필요충분조건은 다음 두 조건이다.

조건 1

모든 대표 원소 p, q 에 대해,

$$\text{pos}_A(p) < \text{pos}_A(q), \quad p > q$$

이면

$$\text{pos}_B(p) < \text{pos}_B(q)$$

이다.

조건 2

대표 원소가 아닌 접미사 최솟값 x 를 생각하자.

x 의 왼쪽에 있는 대표 원소 중 pos_B 의 값이 가장 큰 원소를 y 라고 하자.

그러면 y 가 존재하고,

$$y < x$$

여야 한다.

조건 1의 필요성

조건 1을 만족하지 않는 대표 원소 p, q 가 존재한다고 하자.

수열 A 에서는 p 가 q 보다 앞에 있고 $p > q$ 이지만, 수열 B 에서는 q 가 p 보다 앞에 있다.

두 원소는 모두 대표 원소이므로 합치기 연산으로 없어질 수 없다. 따라서 두 원소의 순서를 바꾸어야 한다.

하지만 p 가 q 의 바로 앞에 놓인 상태에서 $p < q$ 가 성립하지 않으므로, 두 원소를 교환할 수 없다.

따라서 조건 1은 필요하다.

조건 2의 필요성

대표 원소가 아닌 접미사 최솟값 x 를 생각하자.

x 를 없애려면, 어떤 순간에 x 보다 작은 값과 합쳐야 한다. x 보다 작은 모든 값은 처음 수열에서 x 의 왼쪽에 있다.

x 가 없어질 때 남는 더 작은 값을 추적하자. 그 값이 이후에 합치기 연산으로 없어진다면, 그보다 더 작은 값이 대신 남는다. 이 과정을 계속하면, 마지막에는 x 보다 작은 어떤 대표 원소 r 이 남아야 한다.

따라서 x 의 왼쪽에는 적어도 하나의 대표 원소가 존재해야 한다.

이제 x 의 왼쪽에 있는 대표 원소 중 pos_B 가 가장 큰 원소를 y 라고 하자. $y > x$ 라고 가정하자.

처음에 y 는 x 보다 왼쪽에 있고 $y > x$ 이므로, x 가 없어지기 전까지 y 와 x 의 상대적인 순서는 바뀔 수 없다.

x 가 없어지는 순간, x 보다 작은 값은 x 와 인접해야 하므로 y 보다 오른쪽에 있다. 한 번 y 보다 오른쪽으로 이동한 y 보다 작은 값은 다시 y 보다 왼쪽으로 이동할 수 없다.

따라서 마지막에 남는 대표 원소 r 도 y 보다 오른쪽에 있다.

하지만 r 은 처음에 x 의 왼쪽에 있던 대표 원소이고, y 는 그러한 대표 원소 중 pos_B 가 가장 크다. 그러므로

$$\text{pos}_B(r) < \text{pos}_B(y)$$

이고, 최종 수열에서는 r 이 y 보다 왼쪽에 있어야 한다. 이는 모순이다.

따라서 반드시 $y < x$ 여야 한다.

부분문제 7

위의 두 조건을 직접 검사할 수 있다.

모든 두 값 p, q 의 쌍에 대해, 수열 A 와 수열 B 를 선형으로 순회하여 $\text{pos}_A(p), \text{pos}_A(q), \text{pos}_B(p), \text{pos}_B(q)$ 를 찾으면 조건 1을 검사할 수 있다.

값의 쌍은 $\mathcal{O}(N^2)$ 개이고, 각 쌍을 검사하는 데 $\mathcal{O}(N)$ 의 시간이 필요하다.

조건 2는 각 접미사 최솟값마다 왼쪽의 대표 원소들을 직접 조사하면 된다.

조건을 모두 검사한 뒤에는, 부분문제 8에서 설명할 구성 방법을 그대로 사용한다.

전체 시간 복잡도는 $\mathcal{O}(N^3)$ 이다.

부분문제 8

두 단계에 걸쳐 대표 원소가 아닌 값들을 없앤 뒤, 대표 원소들을 수열 B 의 순서대로 정렬한다.

첫 번째 단계

수열 A 를 오른쪽에서 왼쪽으로 순회하면서, 대표 원소가 아니고 접미사 최솟값도 아닌 값들을 없앤다.

현재 처리하는 값을 x 라고 하자. x 는 접미사 최솟값이 아니므로, x 의 오른쪽에는 x 보다 작은 값이 존재한다.

x 의 오른쪽에서 처음 등장하는 x 보다 작은 값을 z 라고 하자. x 와 z 사이의 모든 값은 x 보다 크다.

따라서 다음과 같이 x 를 오른쪽으로 이동시킬 수 있다.

$$[\dots, x, c_1, c_2, \dots, c_k, z, \dots] \rightarrow [\dots, c_1, c_2, \dots, c_k, x, z, \dots]$$

여기서 모든 c_i 는 x 보다 크므로, 사용한 모든 교환 연산은 유효하다.

이후 x 와 z 를 합치면 x 가 없어지고 z 가 남는다.

오른쪽에서 왼쪽으로 처리하기 때문에, 현재 처리하는 위치보다 왼쪽의 원소들은 바뀌지 않는다. 또한 더 작은 값은 합치기 연산으로 없어지지 않으므로, 필요한 z 는 항상 존재한다.

이 단계가 끝나면, 남아 있는 대표 원소가 아닌 값들은 모두 원래 수열 A 의 접미사 최솟값이다.

두 번째 단계

이제 현재 수열을 왼쪽에서 오른쪽으로 순회한다.

대표 원소를 만날 때마다, 지금까지 만난 대표 원소들이 수열 B 의 순서대로 정렬되도록 현재 대표 원소를 삽입 정렬한다.

현재 대표 원소를 q 라고 하고, q 가 왼쪽의 대표 원소 p 를 넘어가야 한다고 하자.

수열 B 에서는 q 가 p 보다 앞에 있어야 한다. 만약 $p > q$ 라면 조건 1에 의해 수열 B 에서도 p 가 q 보다 앞에 있어야 하므로 모순이다.

따라서

$$p < q$$

이고, p 와 q 에 교환 연산을 수행할 수 있다.

대표 원소가 아닌 값 x 를 만났다고 하자. 첫 번째 단계가 끝났으므로 x 는 원래 수열 A 의 접미사 최솟값이다.

x 보다 왼쪽에 있던 대표 원소들은 이미 수열 B 의 순서대로 정렬되어 있다. 따라서 x 의 바로 왼쪽에는, 원래 x 의 왼쪽에 있던 대표 원소 중 pos_B 가 가장 큰 원소 y 가 놓여 있다.

조건 2에 의해

$$y < x$$

이므로, y 와 x 를 합치면 x 만 없어진다.

이 작업을 반복하면 모든 대표 원소가 아닌 값들이 없어지고, 남은 대표 원소들은 수열 B 의 순서대로 정렬된다.

따라서 최종 수열은 정확히 수열 B 가 된다.